

Deployment Guide

1. Prerequisites

- A Contabo account with API credentials (OAuth2 client + account user/password) for Terraform, or an existing Ubuntu 22.04 VPS with root access for manual deployment.
- A domain or nip.io-style wildcard (this deployment: 169-58-221-49.nip.io) for Traefik's automatic TLS no DNS setup needed with nip.io.
- Terraform $\geq$ 1.5.0 (laC path only).
- API keys: VirusTotal, AbuseIPDB, OTX, Shodan, URLScan, plus at least one of OpenAI/Gemini/DeepSeek/Qwen (Ollama needs none — runs locally).

2. Infrastructure Provisioning (Terraform)

The terraform/ directory provisions the VPS via the Contabo provider.

```
export TF_VAR_contabo_client_id="..."
export TF_VAR_contabo_client_secret="..."
export TF_VAR_contabo_user="..."
export TF_VAR_contabo_password="..."
```

These map to variables.tf and are never hardcoded or committed. Key sizing variables: instance_product_id (default V45, ~4 vCPU/8GB RAM), contabo_region (EU), instance_image_id (Ubuntu 22.04 LTS), ssh_key_ids (populate before applying).

```
cd terraform
terraform init
terraform plan
terraform apply
```

outputs.tf prints vps_public_ip, backend_url (:8000), frontend_url (:3000), n8n_url (:5678). What happens automatically: the VPS boots with cloud-init.yml, which installs Docker, adds a 4GB swapfile (for the ~22-container stack on 8GB RAM), clones the repo into /opt/soc, and runs docker compose up -d — a fully running platform with no manual SSH steps.

3. Manual Deployment (existing VPS)

Replicates what cloud-init.yml does by hand: install Docker from the official apt repo, add a 4GB swapfile, then:

```
mkdir -p /opt/soc
git clone https://github.com/Inshalikram/Security-Operations-Platform.git /opt/soc
cd /opt/soc
```

4. Environment Configuration

Backend (backend/.env)

Variable	Purpose
DATABASE_URL	Postgres connection string
KEYCLOAK_URL / KEYCLOAK_INTERNAL_URL	Public and Docker-internal Keycloak URLs — both accepted as valid JWT issuers
VIRUSTOTAL_API_KEY, ABUSEIPDB_API_KEY, OTX_API_KEY, URLSCAN_API_KEY, SHODAN_API_KEY	Threat intel provider keys
THEHIVE_API_KEY, THEHIVE_URL	Case management integration
OPENAI_API_KEY, GEMINI_API_KEY, DEEPSEEK_API_KEY, QWEN_API_KEY / OLLAMA_URL	AI Gateway provider keys (Ollama needs no key)
AI_PROVIDER	Default provider when none specified (default "ollama")
WAZUH_API_URL/USER/PASSWORD, WAZUH_INDEXER_URL/USER/PASSWORD	Wazuh Manager REST + Indexer credentials
API_KEYS	Comma-separated keys accepted by /internal/verify-api-key
OTEL_EXPORTER_OTLP_ENDPOINT	Tempo OTLP endpoint (default http://tempo:4317)

Frontend (frontend/.env.local)

Variable	Purpose
NEXT_PUBLIC_API_URL	Backend base URL the browser calls directly
NextAuth / Keycloak client settings	Configured via auth.ts — client ID/secret registered in the soc-platform realm

5. Starting the Stack

```
cd /opt/soc
docker compose up -d
docker compose ps # confirm all ~22 containers are Up
```

First-boot ordering: wazuh-certs-generator must complete before wazuh-indexer/wazuh-manager start (handled by depends_on); Keycloak needs Postgres up first. If Wazuh fails to come healthy on first boot, re-run docker compose up -d once certs are generated.

6. Post-Deployment Setup

- Keycloak: log in (initial admin/admin), confirm the soc-platform realm, create analyst users, register the frontend client with correct redirect URIs.
- TheHive: create the soc-platform org, generate an API key for a service account, set as THEHIVE_API_KEY.
- Knowledge base: run python seed_knowledge.py once to populate Sigma rules, MITRE ATT&CK, sample CVEs, and playbooks for RAG.
- Elasticsearch backfill (optional): python backfill_search.py to index existing Postgres rows.

- n8n: log in (basic auth), import/configure IMAP, VirusTotal, OTX, Slack webhook, SMTP, and backend OAuth2 credentials, then activate the workflow.
- Grafana: log in (admin/admin123 — rotate immediately), confirm Prometheus/Loki/Tempo data sources and the 7 dashboards are populated.

7. Verifying the Deployment

Check	How
All containers healthy	<code>docker compose ps</code>
Backend reachable	<code>curl https://api.169-58-221-49.nip.io/health → {"status":"healthy"}</code>
Frontend login works	Visit <code>app.169-58-221-49.nip.io</code> , complete a Keycloak login
Threat check end-to-end	Search a known IP; confirm it appears on Dashboard and (if malicious) in Cases
Monitoring tools reporting	Network Status page shows Suricata/Zeek/Wazuh/Falco all "Active"
Traefik routing/TLS	Dashboard at <code>:8081</code> shows all routers/services green with valid certs